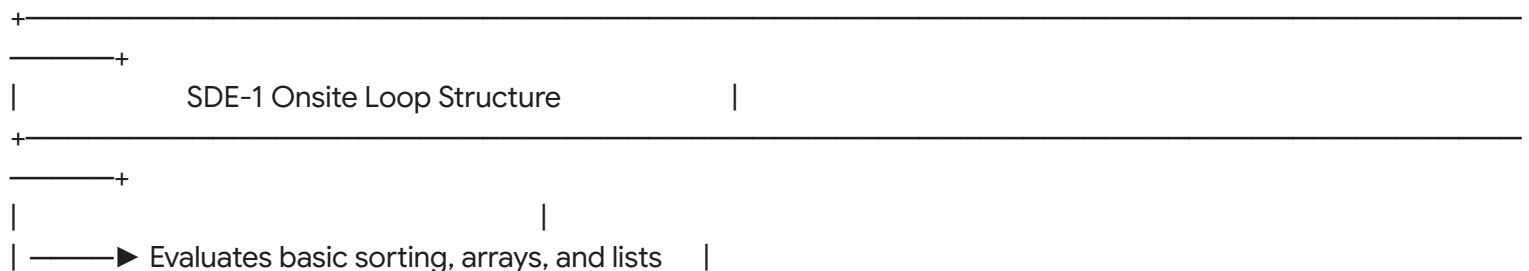
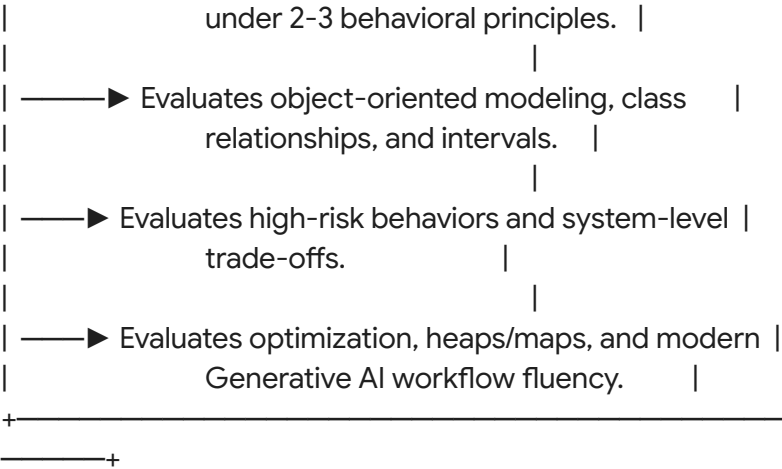


The Macro Architecture and Evolutionary Timeline of the SDE-1 Loop

The recruitment pipeline begins with the Online Assessment (OA), typically hosted on the HackerRank platform.¹ In 2026, the OA evolved beyond standard algorithmic challenges to incorporate interactive development simulations.⁷ Candidates are given 90 to 120 minutes to complete three distinct components: a standard Data Structures and Algorithms (DSA) test, an interactive backend coding exercise featuring an AI-assisted development partner, and a behavioral Work Style Assessment.¹

Following a successful OA, the candidate proceeds to the onsite interview loop.¹ In regions like India, this loop has transitioned back to mandatory in-person evaluation drives.⁸ In other global regions, virtual formats conducted via Amazon Chime remain standard.² The onsite loop for SDE-1 and New Grad roles consists of three to four sequential, 45 to 60-minute interviews.⁴ Each round is led by a different engineer or manager, ensuring that the panel evaluates unique, non-overlapping behavioral and technical competencies.¹





This four-round structure enforces a strict division of labor across the interviewing panel.¹ The specific operational phases of this pipeline are detailed below:

Interview Phase	Technical Focus & Environment	Evaluative Metrics	Timeline
Application ⁵	Applicant Tracking System (ATS). ⁵	Resume screening, additional criteria, and background checks. ¹²	0. ⁵
Assessment (OA) ¹	Rank; 2 DSA problems, 1 backend AI task, and behavioral profiling. ¹	Algorithmic logic, interviewing, and initial assessment. ²	1 to 4 post-application
Round 1 ⁴	(Chime) or In-person; Live whiteboard or collaborative editor. ¹	Algorithmic problem-solving, Bucket Sort, Array-based merging, and clarity of communication.	2 to 10 post-application
Round 2 ⁴	Collaborative editor; Low-Level Design (LLD) problem (e.g., Parking Lot Design). ⁴	Object-Oriented Design patterns, class design, and data structure manipulation under pressure.	Different with Round 1
Round 3 (Bar Raiser) ⁴	(Chime) or In-person; 60% Behavioral, 40% Technical. ²	Behavioral verification, independent decision-making under uncertainty, and structural integrity limits. ²	Independent of Round 2. ⁴

Round 4 (Hiring Manager)	collaborative editor; Focus on optimization and code-quality trade-offs. ⁴	Generative AI integration metrics, memory-vs-latency optimizations (e.g., Maps and Heaps), and operational grit.	Final stage of onsite loop.
---------------------------------	---	--	-----------------------------

Recruitment Phase	Technical Focus & Environment	Core Evaluative Metrics	Target Timeline
Online Application	Digital Applicant Tracking System (ATS).	Resume screening, educational criteria, and basic qualification checks.	Month 0.
Online Assessment (OA)	HackerRank; 2 DSA problems, 1 backend AI-assisted coding task, and behavioral profiling.	Algorithmic logic, interactive debugging, and initial cultural alignment.	Weeks 1 to 4 post-application.
Onsite Round 1	Virtual (Chime) or In-person; Live whiteboard or collaborative editor.	Algorithmic problem-solving (e.g., Bucket Sort, interval-based merging) and clarity of communication.	Month 2 to 10 post-application.
Onsite Round 2	Collaborative editor; Low-Level Design (LLD) schema creation (e.g., Parking Lot Design).	Object-Oriented Design (OOD) patterns, class structure, and data encapsulation under pressure.	Concurrent with Round 1.
Onsite Round 3 (Bar Raiser)	Virtual (Chime) or In-person; 60% Behavioral, 40% Coding.	Deep behavioral verification, decision-making under high ambiguity, and structural scalability limits.	Scheduled independently or back-to-back.
Onsite Round 4 (Hiring Manager)	Collaborative editor; Focus on optimization and code-quality trade-offs.	Generative AI integration metrics, memory-vs-latency optimizations (e.g., Maps and Heaps), and operational grit.	Final stage of onsite loop.

A notable addition to the 2026 onsite loop is "Gen AI Fluency".⁴ Rather than simply evaluating abstract coding ability, hiring managers assess how candidates integrate AI code-generation utilities into their daily work.⁴ Candidates must explain their interaction with AI tools and provide metrics that quantify the impact on their coding productivity and speed.⁴

The Internal Grading Continuum: Above Bar versus At Bar Rubrics

The evaluation of an SDE-1 candidate is governed by an objective assessment matrix.¹⁴ Interviewers rate candidates across defined functional and behavioral competencies on a strict scale of "Above Bar," "At Bar," or "Below Bar".¹⁵ However, this rating does not automatically determine the candidate's hiring outcome.¹⁵ The competency-based bar rating and the final recommendation to "Incline" or "Not Incline" are handled as separate decisions.¹⁵ An interviewer can evaluate a candidate as "At Bar" or "Above Bar" across all technical competencies and still issue a "Not Incline" vote.¹⁵ This occurs when the interviewer detects qualitative issues that the standard rubrics cannot fully capture.¹⁵ These issues include a lack of communication clarity, defensiveness when receiving feedback, or an artificial, rehearsed presentation.¹⁵ Conversely, a candidate who drops "Below Bar" on a single competency can still receive a hiring recommendation if

they demonstrate "Above Bar" capabilities in other high-priority areas.¹⁵ For SDE-1 and New Grad candidates, the evaluation centers on five core technical and behavioral dimensions¹⁴:

Dimension	Requirements (SDE-1 Base)	Bar Performance Indicators (\$
Scope & Influence ¹⁴	successfully delivers well-defined, task-oriented assignments within a single software development cycle. ¹⁴	understands upstream and downstream dependencies; demonstrates influence beyond their direct reporting line. ¹⁴
Ambiguity ¹⁴	clarifies requirements effectively once system requirements and technical designs are clearly established.	achieves progress in vague environments with incomplete data. ²
Technical Complexity ¹⁴	solves standard algorithmic and logical problems; produces clean, modular, and reusable code. ¹	demonstrates a deep understanding of system architecture and software performance trade-offs; implements advanced data structures.
Execution ¹⁴	completes individual assignments on schedule; identifies and escalates project blockers.	proactively identifies edge cases, develops comprehensive testing structures, and refactors code. ⁹
Impact ¹⁴	implements localized fixes and builds features that deliver immediate value to the software system.	implements structural optimizations that improve system latency, resource consumption, and scalability. ¹

New Grad candidates, the evaluation centers on five core technical and behavioral dimensions.

Rubric Dimension	At Bar Requirements (SDE-1 Base)	Above Bar Performance Indicators (SDE-2 Signal)
Scope & Influence	Successfully delivers well-defined, task-level requirements within a single software component.	Understands upstream and downstream architecture boundaries; demonstrates initial system influence beyond their direct assignment.
Ambiguity	Operates effectively once system requirements and technical designs are clearly established.	Successfully gathers requirements and makes technical progress in vague environments with incomplete data.
Technical Complexity	Solves standard algorithmic and logical challenges; produces clean, modular, and readable code.	Demonstrates a deep understanding of hardware and software performance trade-offs; implements advanced data structures.
Execution	Delivers individual assignments on schedule; identifies and escalates project blockers early.	Proactively identifies edge cases, develops automated testing structures, and refactors legacy code.
Impact	Delivers localized fixes and builds features that add immediate value to the software system.	Implements structural optimizations that reduce overall system latency, resource consumption, or cost.

The primary hurdle that prevents SDE-1 candidates from receiving "Above Bar" ratings—and subsequently delays their promotion to SDE-2—is a lack of technical complexity in their work.¹⁴ While many early-career engineers can execute tasks on schedule, they often struggle to design and evaluate solutions under real-world constraints.⁴ To simulate these rigorous internal rubrics, expert interview coaches utilize structured mock evaluations.¹⁷ Nick Dimitrov's "Amazon Bound" coaching framework uses a mock process consisting of a one-hour simulation, followed by one hour of independent, behaviorally justified assessment writing by a former Bar Raiser, and a final one-hour live debrief.¹⁷ This methodology evaluates candidates using a proprietary, data-driven scoring system on a scale of 1 to 100¹⁷:

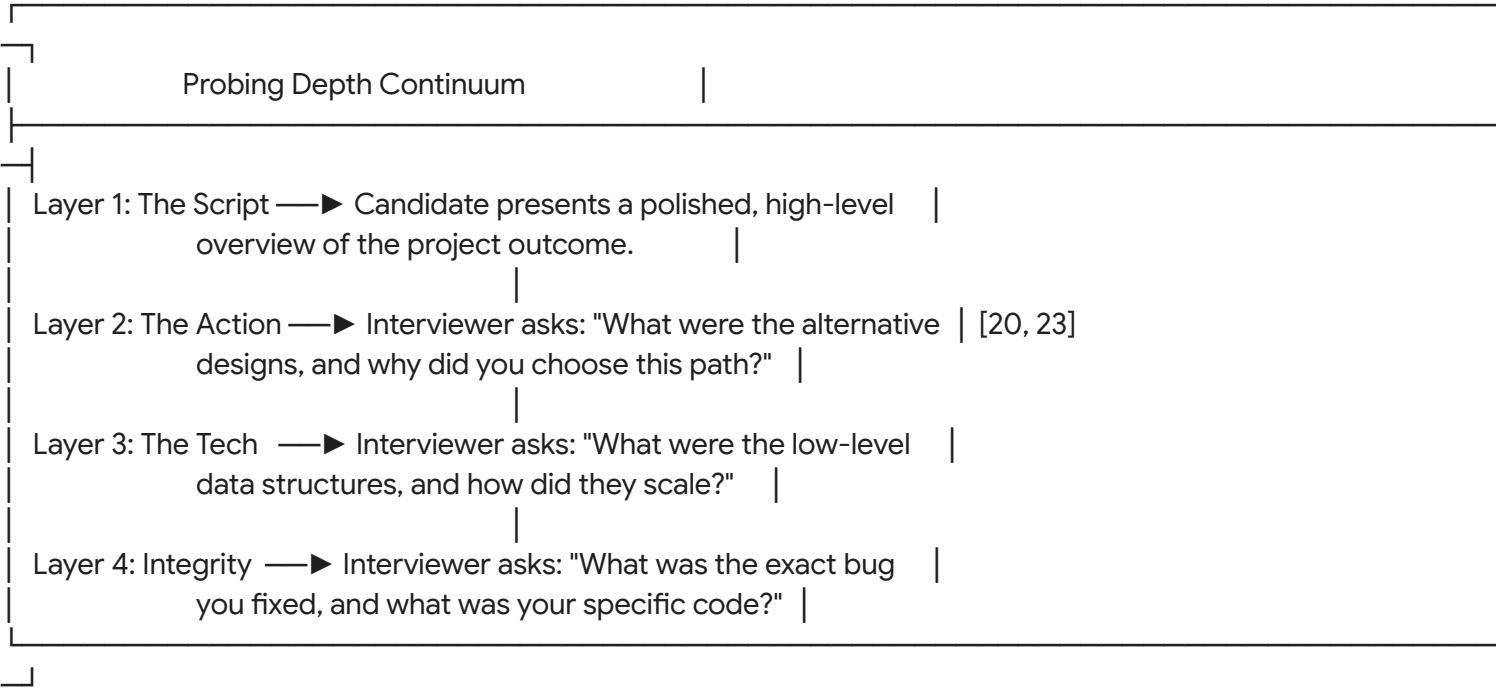
$$\text{Candidate Evaluation Score} \in$$

A score between 1 and 49 indicates a "Needs Improvement" rating.¹⁷ This score is assigned when a candidate's STAR behavioral stories are missing key structural elements, fail to show clear individual impact, or lack the quantitative metrics required by Amazon's data-driven culture.⁸

Behavioral Probing Strategies and the Forensic Anatomy of Story Deception

The behavioral interview loop is designed to look past rehearsed scripts and uncover the actual impact of a candidate's past work.¹⁶ Interviewers are trained to conduct forensic questioning to determine whether the candidate's achievements are genuine.¹¹ According to experienced Bar Raiser coaches like Aimee, candidates frequently fail behavioral evaluations due to four common pitfalls¹⁶:

- **Not Enough Individual Impact:** Candidates often use collective pronouns like "we" when describing project achievements.¹⁶ This signals to the interviewer that the candidate cannot isolate their personal contribution or may be taking credit for others' work.¹
- **Crumbling Under Deep Probing:** Many candidates rely on rehearsed scripts and struggle when asked detailed follow-up questions.¹⁶ This vulnerability is exposed when the interviewer probes the "Action" phase of their story.¹⁶
- **Theme Inconsistencies:** During the final debrief, interviewers compare candidate stories to verify their consistency.⁸ Telling conflicting stories or tailoring the facts of a project to fit different leadership principles raises immediate concerns about the candidate's integrity.⁸
- **Not Operating at the Next Level:** Amazon looks for early-career candidates who demonstrate the potential to think and act strategically.¹⁶ This is shown when a candidate can influence peers or resolve team conflicts without relying on formal authority.¹⁶



To systematically uncover these signals, experienced interviewers use specific leadership principles to guide their probing.²³ This forensic approach is outlined below:

Relationship Principle	Competency Focus	Probing Question	Red Above-Bar Signal
Senior Obsession ¹⁶	Senior empathy, priority-setting, and balancing user needs against technician needs.	<i>...about a time you had to choose between the immediate needs of the user and the long-term health of the system."</i> ²⁵	candidate shows how they use metrics to drive technical decisions, choosing long-term over a quick, temporary
Senior Action ¹⁶	risk mitigation, and weighing in on reversible vs. irreversible choices. ²⁷	<i>...about a time you shipped with known technical debt and decided that risk was acceptable?"</i> ²⁷	candidate demonstrates a standing of "two-way doors," using automated metrics and feedbacks to manage calculated
Senior Deep ¹⁶	systematic debugging, root-cause analysis, and data-driven decision-making decisions. ²⁵	<i>...about a complex bug that occurred in production. How did you determine the root cause when the logs were unclear?"</i> ²²	candidate explains their methodology, detailing memory usage, or network packets with technical depth. ⁴
Senior Trust ¹⁶	willingness to constructive feedback, including peer disagreements and admitting mistakes. ¹⁶	<i>...about a time you received feedback from a senior engineer and took specific actions to address it?"</i> ¹⁶	candidate is vocally self-critical, taking responsibility for technical mistakes and implementing clear feedback loops to improve performance. ⁹

Leadership Principle	Target Competency Focus	Key Probing Question	Expected Above-Bar Signal
Customer Obsession	Core user empathy, priority-setting under pressure, and balancing customer needs against technical debt.	<i>"Tell me about a time you had to balance the immediate needs of a customer with the long-term health of your system."</i>	The candidate shows how they used customer metrics to drive technical architecture, choosing long-term stability over a quick, temporary fix.
Bias for Action	Speed, risk mitigation, and distinguishing between reversible and irreversible choices.	<i>"Tell me about a time you shipped a feature with known technical debt. How did you decide that risk was acceptable?"</i>	The candidate demonstrates a clear understanding of "two-way door" decisions, using automated monitoring and rollbacks to manage calculated risks.
Dive Deep	Systematic debugging, root-cause identification, and data-driven engineering decisions.	<i>"Tell me about a complex bug that occurred in production. How did you locate the root cause when the metrics were unclear?"</i>	The candidate explains their debugging methodology, detailing memory layouts, trace logs, or network packets to show their technical depth.
Earn Trust	Openness to constructive feedback, managing peer disagreements, and willingness to admit mistakes.	<i>"Tell me about a time you received critical feedback from a senior engineer. What specific actions did you take to address it?"</i>	The candidate is vocally self-critical, taking responsibility for technical mistakes and implementing clear feedback loops to improve their performance.

Senior interviewers often use multi-layered behavioral questions to evaluate several leadership principles at once.¹⁶ These questions are designed to make it difficult for candidates to rely on rehearsed, single-principle scripts.¹⁶ For example, Diana, a Senior Product Manager, utilizes the prompt, *"Tell me about a time when you had significant obstacles delivering a project,"* to simultaneously evaluate *Deliver Results*, *Ownership*, *Disagree and Commit*, and *Customer Obsession*.¹⁶ Anurag, an engineering leader, uses, *"Tell me about a time when you had to deliver something with very limited resources or tight constraints,"* to evaluate *Frugality*, *Invent and Simplify*, *Bias for Action*, and *Ownership* under pressure.¹⁶ Additionally, Artiom, a Senior Frontend Engineer, probes deeply into *Ownership*, *Deliver Results*, *Earn Trust*, and *Dive Deep* by asking: *"Tell me about a time when you realized you were not able to meet a commitment on a long-lasting project or initiative. How did you navigate the situation?"*¹⁶

Translation of Non-Traditional and Early-Career Experiences

New Grad and SDE-1 candidates often lack extensive enterprise experience, typically entering the loop with academic capstone projects, volunteer work, or internship portfolios.⁴ The challenge for these candidates is translating their projects into professional, data-driven narratives that align with Amazon's core leadership principles.¹¹

Experience Translation Matrix	
Academic Capstone	—► Translate "team grade" into "individual code ownership and algorithmic complexity"
Open-Source	—► Translate "feature pull requests" into "customer impact and dependency management"
Service-Based Co.	—► Translate "assigned client ticket execution" into "proactive refactoring and scaling metrics" [5, 34]

This translation requires moving past high-level project overviews to focus on individual technical decisions and outcomes.¹¹ The process of translating early-career work into high-impact STAR narratives is detailed below:

1. The Capstone Project Translation

Early-career candidates often describe academic assignments in broad, collective terms (e.g., "Our team designed a mobile application for local businesses").¹⁶ This approach fails to demonstrate individual engineering capability.¹⁶ The candidate must instead focus on their personal contributions, isolating the specific features, data structures, or optimization work they owned.⁹

- *Ineffective Academic Phrasing:* "We built a web application using React and Node.js, and used a PostgreSQL database to manage user profiles".¹
- *Effective Professional Translation:* "I designed the user authentication flow and database schema.¹⁶ To protect user data, I replaced a basic database search with an indexed query, which reduced authentication API response times from **180ms** to **32ms**, representing an **82.2%** latency improvement".¹

2. The Internal Customer Paradigm

New Grads frequently assume that *Customer Obsession* only applies to external consumers.²⁴ For platform, backend, or infrastructure engineers, the customer is often another internal team, developer, or downstream API consumer.²³

Early-career candidates must frame their engineering decisions around how they supported and enabled these internal clients.²³

- *Ineffective Technical Phrasing:* "I refactored our team's data ingestion library because the legacy code was difficult to read and maintain".¹
- *Effective Customer-Centric Translation:* "I identified that our peer development teams were spending an average of two hours configuring our data ingestion library due to a lack of documentation.²⁴ I designed a simplified

API wrapper that reduced setup time to under ten minutes, eliminating integration issues and allowing three downstream teams to ship their features ahead of schedule".²³

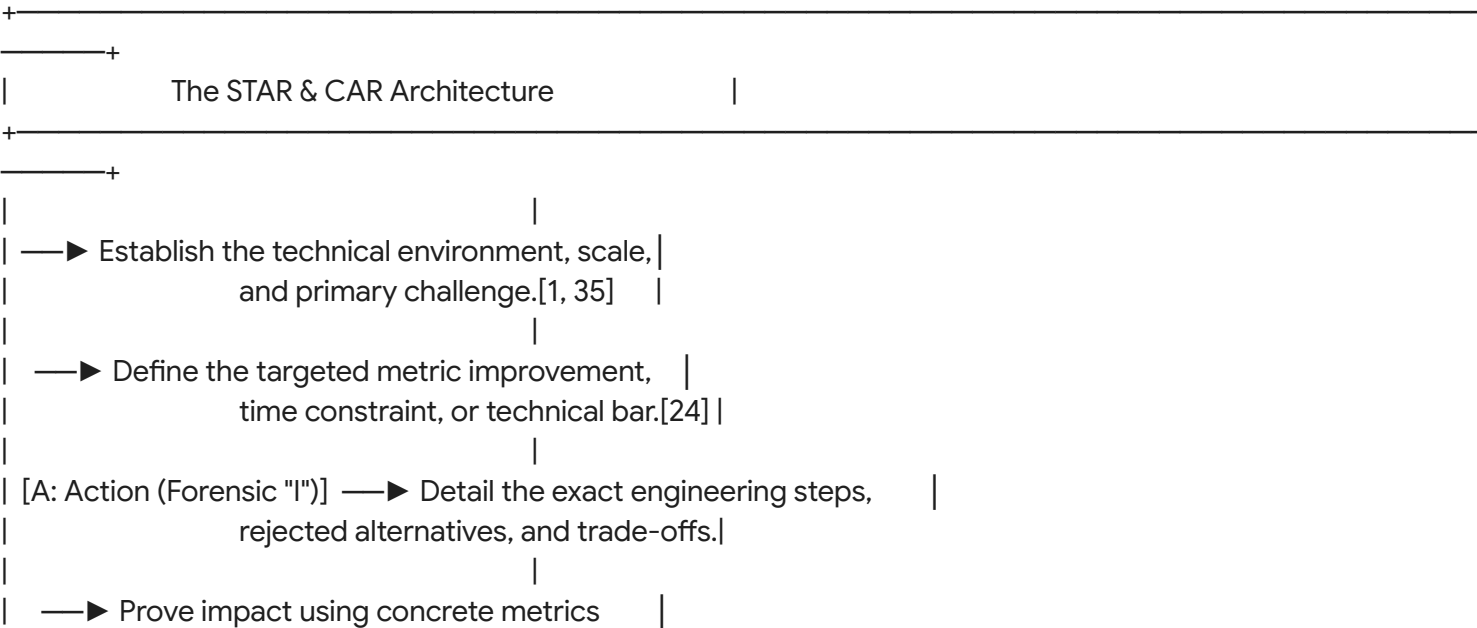
3. The Volunteer and Open-Source Translation

Volunteer and open-source contributions are valuable sources of behavioral and technical signals, but they are often described informally.²⁴ Candidates should treat these contributions as professional engineering projects, focusing on system scale, code quality, and collaboration.⁹

- *Ineffective Informal Phrasing:* "I volunteered to help fix issues and update the user interface for a local non-profit website".²⁴
- *Effective Professional Translation:* "I took ownership of a performance issue on a volunteer platform that was causing page timeouts during traffic spikes.³² I monitored system resource usage and discovered a memory leak caused by unclosed database connections.⁹ I refactored the connection lifecycle management, which reduced server CPU usage by 45% and eliminated page timeout errors during high-traffic events".⁸

The Pivot Strategy: Optimizing the STAR and CAR Narrative Portfolio

The behavioral interview loop evaluates candidates through structured, narrative-based frameworks.³ The standard framework is the STAR method (Situation, Task, Action, Result), which is closely related to the CAR method (Context, Action, Result).³



| (latency, CPU, throughput, errors). |

+

+

To prepare effectively, candidates should use the "Pivot Strategy," where they build five multi-dimensional, high-quality stories rather than trying to memorize a separate story for every leadership principle.²³ This strategy ensures that each story can be adapted to highlight different principles depending on the interviewer's focus, while preventing the candidate from repeating the same story across different rounds.⁸ This narrative matrix is mapped below, showing how five core stories can cover twelve key leadership principles:

Story Theme	Primary LP Alignment	Secondary LP Alignment	Technical Element	Leadership Strategy (TIP)
Complex Bug ²³	Deep ²³	Learn & Be Curious ³⁶	Memory leaks, race conditions, logging, or network statistics. ⁹	<i>Learn & Be Curious</i> by showing how you diagnosed the issue ²⁸ ; pivot to <i>Take Action</i> by detailing the various mitigations we used to protect users
Technical Disagreement	Backbone; Disagreement ²³	Trust ²³	Algorithmic pattern choice, data structure tradeoffs, or API design.	<i>Earn Trust</i> by showing how you listened to the peer's perspective; pivot to <i>Are Right, A Lot</i> by explaining how data and process were used to resolve the conflict.
Tight Deadline ¹⁶	Take Action ²⁷	Frugality ²⁷	Cutting down features, choosing dependencies, or optimizing infrastructure was a tradeoff. ²⁷	<i>Frugality</i> by explaining how infrastructure was optimized; pivot to <i>Save Costs</i> by detailing new costs ²⁷ ; pivot to <i>Show Results</i> by highlighting milestones were met.
Technical Setback ¹⁶	Ownership ²³	Insist on High Standards	Deployment failures, broken builds, or missing dependencies.	<i>Insist on High Standards</i> by showing how automated tests were improved to prevent the issue ⁹ ; pivot to <i>Own It</i> by explaining how the candidate took responsibility for the response. ²²
Availability Project ²³	Customer Obsession ²³	Show Results ²³	Service outages, query optimization, or caching. ²	<i>Customer Obsession</i> by showing how the change improved the end user experience.

				ence ²³ ; pivot to <i>Del</i> by focusing on the input improvements
--	--	--	--	--

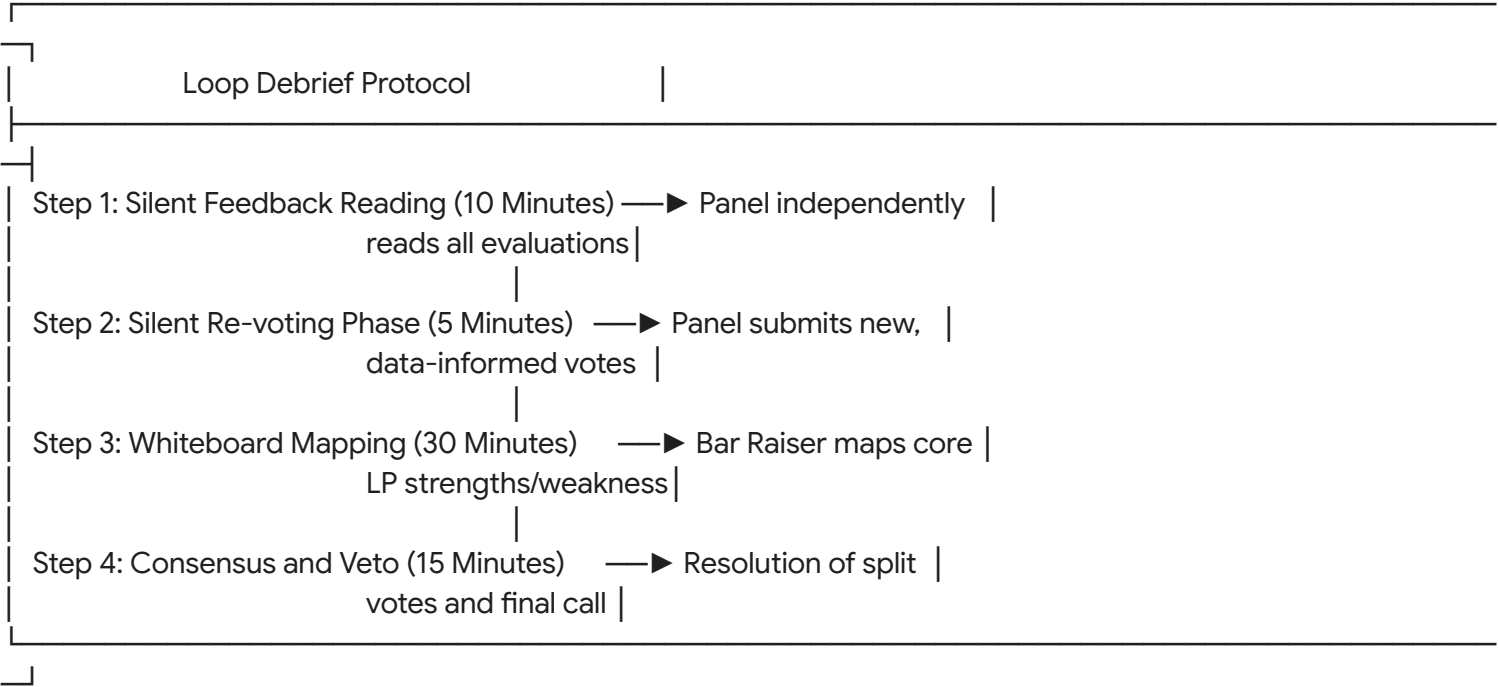
Core Story Theme	Primary LP Alignment	Secondary LP Alignment	Targeted Technical Elements	Adaptability Strategy (The Pivot)
The Complex Bug	Dive Deep	Learn & Be Curious	Memory leaks, race conditions, trace logging, or network diagnostics.	Pivot to <i>Learn & Be Curious</i> by focusing on the new tools learned to diagnose the issue ; pivot to <i>Bias for Action</i> by detailing how temporary mitigations were deployed to protect users.
The Technical Disagreement	Have Backbone; Disagree & Commit	Earn Trust	Design pattern choice, database schema tradeoffs, or API styling.	Pivot to <i>Earn Trust</i> by showing empathy for the peer's perspective ; pivot to <i>Are Right, A Lot</i> by showing how data and prototypes were used to resolve the conflict.
The Tight Deadline	Bias for Action	Frugality	Scoping down features, choosing dependencies, or optimizing compilation.	Pivot to <i>Frugality</i> by explaining how existing infrastructure was reused to avoid new costs ; pivot to <i>Deliver Results</i> by highlighting how project milestones were met.
The Technical Setback	Ownership	Insist on High Standards	Deployment failures, broken production builds, or missing data.	Pivot to <i>Insist on High Standards</i> by showing how automated testing was improved to prevent recurrence ; pivot to <i>Ownership</i> by explaining how the candidate ran the response.
The Scalability Project	Customer Obsession	Deliver Results	Microservices, query optimization, indexing, or caching.	Pivot to <i>Customer Obsession</i> by explaining how the changes directly improved the end-user experience ; pivot to <i>Deliver Results</i> by focusing on the system throughput improvements.

This matrix allows early-career candidates to prepare a focused set of deeply understood stories.²³ This depth is critical because during the loop, interviewers will ask detailed questions about implementation decisions, technical constraints, and alternative approaches.⁴

The Mechanics of the Bar Raiser Debrief and Veto Resolution

The final decision-making step in the recruitment process is the loop debrief, which is designed to protect the hiring bar and eliminate bias.¹² The debrief is managed by the Bar Raiser, a neutral interviewer who does not belong to the hiring team's organization.¹

The meeting follows a strict, time-managed agenda to ensure all feedback is evaluated objectively ⁴²:



The "re-vote" is a critical part of the process.⁴² Interviewers often adjust their initial votes after reading the detailed technical and behavioral findings compiled by their colleagues.⁴²

When there are conflicting perspectives on the panel, the Bar Raiser coordinates a systematic discussion.⁴² They write two columns on a whiteboard—reasons to hire and reasons not to hire—categorized by leadership principles.⁴² The Bar Raiser then leads the panel to evaluate the candidate's performance against the bar by asking: *"Does anyone have concrete data that suggests a principle belongs in a different column?"*⁴²

The Bar Raiser's primary responsibility is to prevent "desperation hiring," where teams lower their standards because they are understaffed and need to fill a position quickly.⁴³ This is a common risk because hiring managers face direct operational pressure to deliver features and maintain their systems.⁴³

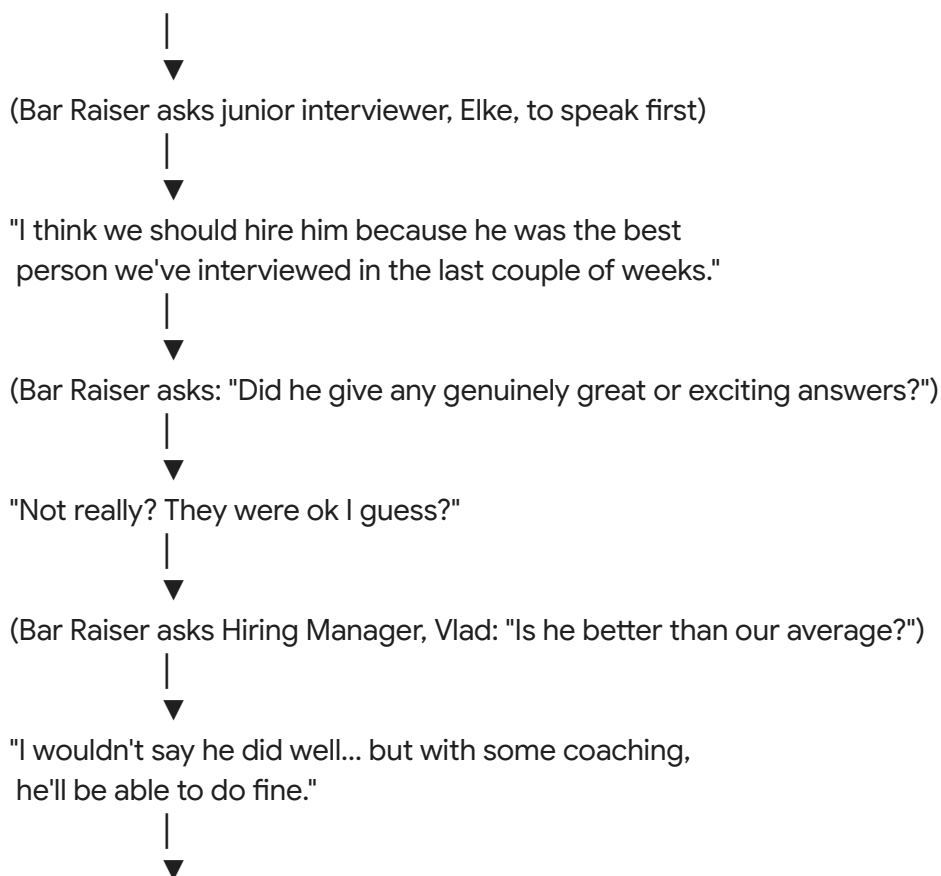
Because the Bar Raiser is a neutral party with no operational connection to the hiring manager's group, they can evaluate the candidate objectively, prioritizing long-term quality over short-term staffing needs.⁴³ The Bar Raiser holds a formal veto, meaning they can block a hire even if the hiring manager and the rest of the loop vote to incline.¹

This dynamic is demonstrated by a real-world case study involving a recruitment loop for an economic scientist candidate named Connor.⁴³ In this case, the hiring team was severely understaffed and had interviewed seven candidates over a multi-week period.⁴³ Connor's actual performance was mediocre—described as a "C minus"

level—and the Bar Raiser, Dave Anderson, voted "not inclined".⁴³

However, the other four interviewers voted "inclined".⁴³ Their written feedback revealed a willingness to settle for a passable candidate, with comments such as: *"He didn't know Y or Z which was disappointing, but his answer was fine... I'll call it good enough"*.⁴³

To expose this compromise during the debrief, the Bar Raiser used a structured questioning approach⁴³:



This case study shows that the hiring bar is defined relative to the current workforce: a candidate must be better than **50%** of the existing employees at that level.¹¹ If the team's excitement is low and the candidate is only expected to "do fine with coaching," they do not meet the bar and must be rejected.⁴³ Even when the hiring manager, Vlad, became highly argumentative—stating that the loop had agreed and the Bar Raiser should get out of their way—the Bar Raiser maintained their veto, and the escalation was rejected by corporate leadership.⁴³

Beyond candidate loops, the Bar Raiser veto can also apply during post-loop reviews if new, high-risk operational data emerges.⁴⁰ For example, in an internship conversion review, an intern manager supported a conversion offer, but the Bar Raiser vetoed the hire because the intern had "caused a Correction of Errors (COE)" incident during their internship.⁴⁰ This outcome demonstrates that causing a major operational incident indicates a failure to meet the *Ownership* and *Insist on High Standards* bars, resulting in an immediate veto during the debrief.²⁶

Synthesized Recommendations for Loop Success

Based on this technical analysis of the SDE-1 loop, several key strategies emerge for candidates preparing for the evaluation process:

- **Isolate Personal Impact with "I" Statements:** Candidates must ensure their behavioral stories focus clearly on their individual contributions.¹⁶ They should explicitly explain the code they wrote, the specific designs they chose, and the exact tests they ran, avoiding collective descriptions of team activities.¹
- **Provide Quantifiable Performance Metrics:** Every STAR and CAR story must include concrete data.⁸ Candidates should quantify the technical and business impact of their work using clear metrics, such as millisecond latency reductions, CPU optimization percentages, or integration time savings.¹
- **Verify Core Algorithmic and Structural Details:** Candidates must be ready to defend their technical decisions under deep probing.⁴ They must be able to explain their choice of data structures, analyze algorithmic complexity, and discuss how their systems scale under heavy production loads.²
- **Maintain Performance Consistency Across the Loop:** Candidates should avoid over-preparing only a few stories.⁴¹ They need to build a balanced portfolio of five multi-dimensional narratives that cover all core leadership principles, ensuring consistent quality throughout the entire loop.⁸

Reference Interview Walkthroughs:

1. (The interview process for the Amazon Software Development Engineer (SDE) New Grad role took a little under three months end-to-end. It followed a fairly standard Amazon pattern that aligns with recent Glassdoor reports for SDE/Software Engineer roles: Online Assessment (skills test + work style/personality test), one technical screen via video, and then a multi-round virtual onsite loop including a bar raiser. Timeline: - Week 0: Applied through the Amazon Company Website for the SDE New Grad role in Seattle. - Week 1: Received recruiter outreach via email and scheduled a 30-minute introductory call for later that week. - Week 2: Completed the recruiter call and received the Online Assessment (OA) link within a couple of days. - Week 3: Completed OA (OA1 + OA2 + Work Style Assessment) in one sitting. - Week 5: Informed by the recruiter that I passed the OA; scheduled a 45-minute technical phone/video interview over Amazon Chime. - Week 6: Completed the technical screen. - Weeks 7–8: Some delay due to interviewer availability; then invited to the final loop. - Week 9: Completed a 3-round virtual onsite loop (3 x 60-minute interviews happening back-to-back with short breaks). A bar raiser participated in the last round. - Week 12: Received final decision via email and a short debrief call with the recruiter. Details of

each component: 1) Recruiter Screen (30 minutes, Amazon Chime) - Focus: Background, resume walk-through, basic understanding of role, preferred locations, and immigration status. - No live coding. High-level questions about experience with Java, data structures, and familiarity with AWS (e.g., EC2, S3) based on my internship. - Tools: Amazon Chime for video/audio. - Outcome: Explanation of the process: Online Assessment → Technical Screen → Virtual Onsite (3 rounds including bar raiser), consistent with Glassdoor's reported stages (skills test, phone interview, one-on-one interviews, and a personality/work style test. 2) Online Assessment (OA) - 2 hours total - Delivered via Amazon's proprietary OA platform linked from an email. - OA1 (Debugging/Logic): - Duration: ~35-40 minutes. - Content: 5-7 debugging questions in Java/C++ style code, plus logical/math problems. Required identifying and fixing bugs in provided code snippets under time pressure. - OA2 (Coding + Simulation): - Duration: ~90 minutes. - Content: Two LeetCode-style medium to medium-hard coding problems (arrays/intervals and DP/greedy), auto-graded with hidden test cases. One problem resembled a typical interval merging question; the other was a job scheduling/max profit variant. - Work Style Assessment: - Duration: ~15-20 minutes integrated at the end of OA2. - Format: 30-40 multiple-choice questions assessing alignment with Amazon's 16 Leadership Principles (e.g., Ownership, Bias for Action, Dive Deep). This matched the personality/work style test mentioned in Glassdoor data. 3) Technical Phone/Video Interview (45 minutes, Technical Round) - Timing: About two weeks after OA completion. - Tool: Amazon Chime with a shared online code editor. - Structure: - 5 minutes: Intros and quick background questions. - 30 minutes: One main coding problem (string/array based, LeetCode medium difficulty). Required walking through a brute-force solution and then optimizing to a sliding-window approach with HashMap and $O(n)$ complexity. - 5-10 minutes: CS fundamentals (hash maps, memory model) and 1-2 short behavioral questions tied to Leadership Principles. - The interviewer evaluated problem-solving approach, clarity of communication, and code correctness rather than just final output. 4) Virtual Onsite Loop (3 rounds, each 60 minutes) - Onsite Interview - Conducted over Amazon Chime as a virtual onsite, consistent with recent Glassdoor reports that many Amazon "onsites" are now remote but still treated as full loops. - Time between scheduling and loop: ~2 weeks. Round 1 - DSA/Algorithms (60 minutes) - Format: One main coding problem plus minor follow-ups. - Question type: Medium difficulty grid/graph problem (count islands) involving DFS/BFS and careful handling of state (visited matrix). - Evaluation: Code quality in Java, handling of edge cases, time/space complexity explanation, and ability to discuss iterative vs recursive approaches. Round 2 - Low-Level Design + Coding (60 minutes) - Format: Hybrid of low-level design and implementation. - Question type: Design and implement an LRU cache with $O(1)$ get/put operations and eviction when at capacity. - Discussion included: - Choice of data structures (HashMap + doubly linked list) and why. - Potential extensions to a distributed cache scenario (e.g., using Amazon ElastiCache or

Redis, sharding, consistent hashing). - Handling concurrency at a high level (locks vs lock-free, but not deeply implemented). Round 3 – Bar Raiser (Behavioral + Coding) (60 minutes) - Interviewer: Senior engineer acting as bar raiser. - Structure: ~35–40 minutes behavioral, ~20–25 minutes coding. - Behavioral: Deep dive into multiple Amazon Leadership Principles, asking follow-up questions to test consistency and depth. Required comprehensive STAR-form answers. - Coding: An intervals-based LeetCode medium problem (inserting and merging intervals) with discussion on scaling to large datasets and thorough testing strategy. Common Tools and Practices: - Amazon Chime for all video interviews. - Browser-based plain-text style coding editor (no autocomplete, minimal syntax highlighting). - Internal OA platform for skills test, debugging, and work style assessment. - Strong emphasis on Leadership Principles in every round, especially in the loop and bar raiser. Assignments/Tests: - Online Assessment served as the primary skills test and personality test. - No separate take-home project: all coding was live during OA and interviews. Overall, the process lined up closely with Glassdoor's descriptions for Amazon SDE roles: skills test as OA (including debugging and DSA questions), phone/Chime interview, one-on-one technical interviews in the loop, personality/work style assessment, and a bar raiser round. Communication between stages was generally clear but there were noticeable gaps (1–3 weeks) between OA, technical screen, and final decision, reflecting the multi-week to multi-month timelines reported by other candidates.

2. (interview on Jan 2026.

I applied for the Amazon Software Development Engineer (SDE) New Grad role in Seattle in early January 2026. I knew from Glassdoor that the process typically spans 1–3 months, involves an online assessment (OA) followed by 3–4 technical/behavioral rounds, and is rated around 3.4–3.7/5 in difficulty. To prepare, I spent about 6 weeks doing LeetCode (mostly medium and some hard), focusing on arrays, strings, trees, graphs, dynamic programming, and intervals. I also reviewed core CS fundamentals (Big-O, hashing, concurrency basics), and read through Amazon's 16 Leadership Principles, creating 2–3 STAR stories for each. For system design, I followed common SDE new grad material (high-level design of URL shortener, news feed, and rate limiter). I used ChatGPT for technical preparation and mock behavioral answers, and GitHub Copilot inside VS Code to practice clean coding patterns. The process started with a recruiter outreach email after I applied on the Company Website. The recruiter call was 30 minutes and mostly covered my background, work authorization, preferred locations, compensation expectations, and a brief overview of the process (OA → one technical screen → virtual onsite loop with 3 rounds including a bar raiser). There were no coding questions on this call but I was asked high-level about my experience with Java and AWS services (mainly EC2 and S3) from an internship. About a week later, I received the OA link. The OA had two parts: OA1 was debugging/logic questions and OA2 was two LeetCode-style coding questions plus a Work Style Assessment. OA1 had 7 small debugging problems in Java and some logic/math questions; I had to fix bugs in code snippets within a time limit. OA2 was 90 minutes: the first question was a medium array/intervals problem (merging overlapping intervals and then answering queries), and the second was a harder DP/greedy hybrid on picking non-overlapping jobs with maximum

profit. The environment auto-judged my code with hidden test cases. After the coding portion, the Work Style Assessment asked 30–40 multiple-choice questions mapping to leadership principles like Ownership, Bias for Action, and Dive Deep. I used ChatGPT beforehand to brainstorm how to interpret the LPs but answered the assessment honestly. About 10 days after completing the OA, I heard back that I passed and was invited to a 45-minute virtual technical interview over Amazon Chime. The interviewer was a mid-level SDE based in Seattle. We used a shared online coding editor (similar to CollabEdit) with no IDE features. After a quick intro, he gave me a string-based LeetCode-style question: given a string, find the length of the longest substring without repeating characters and return the substring itself. I walked through a brute-force $O(n^2)$ solution, then moved to an optimized sliding window with a HashMap for last-seen indices in $O(n)$ time. I narrated my thought process and wrote code in Java. He asked follow-ups about edge cases (empty string, all same character, Unicode), and then we briefly discussed time/space complexity. After that, he asked a couple of CS fundamentals questions about hash maps (collision handling, load factor) and memory (stack vs heap). The last 10 minutes were behavioral: I got one LP question around “Tell me about a time you had to dive deep into a production issue” and I used a STAR story from my internship where we debugged a memory leak in a Java microservice using AWS CloudWatch metrics and heap dumps. The interviewer was engaged, occasionally prompted me when I got stuck, and gave subtle hints but didn’t give away the solution. Two weeks later, the recruiter emailed me to schedule the final onsite loop (virtual onsite) with three 1-hour rounds on the same day. There was a small scheduling delay due to interviewer availability, so the loop happened about a month after the OA. The loop consisted of: one DSA-focused coding round, one mixed low-level design/technical round, and one bar raiser that was heavy on Leadership Principles plus a medium coding question. In the first loop round, the interviewer (another SDE) gave me a medium tree/graph problem: given a 2D grid of 0s and 1s, count the number of islands (connected components of 1s using 4-directional adjacency). I explained both DFS and BFS solutions, discussed recursion depth versus an explicit stack/queue, then implemented DFS in Java. We then discussed optimization and trade-offs in using recursion vs an iterative approach to avoid stack overflow on large grids. The second round was more of a low-level design plus coding round. I was asked to design a simplified in-memory key-value cache with eviction, supporting get and put with $O(1)$ time complexity and LRU eviction when capacity is exceeded. I proposed an architecture using a combination of a HashMap and a doubly linked list. I drew the design verbally and in the shared editor, implemented the Node class and the LRUCache class in Java, and handled edge cases. We also briefly talked about how this would differ if it were a distributed cache using something like Amazon ElastiCache or Redis: consistent hashing, sharding, and cache invalidation strategies. The final round was with a bar raiser (a senior engineer with many years of experience at Amazon). It was about 60% behavioral, 40% coding. I was asked several deep LP questions: examples around Ownership, Insist on the Highest Standards, and Have Backbone; Disagree and Commit. For each, I used the STAR format and tried to quantify impact (e.g., reduced error rate by X%, improved latency by Y ms). The coding portion was a variant of an intervals problem: given a set of intervals, insert a new interval and merge any overlapping intervals. I talked through sorting, merging logic, and implemented a clean solution. The bar raiser pushed on details: how would this scale with millions of intervals, what if intervals stream in real-time, and how would I test this thoroughly. Throughout the process, there was minimal live feedback; interviewers wouldn’t indicate pass/fail signals. They were generally professional, though the bar raiser was more intense and often asked follow-up questions to test depth. The main challenges for me were managing time during OA2, staying calm while coding in a plain text editor without autocomplete, and keeping all my LP stories fresh in mind during the long loop day. I also found the uncertainty between rounds stressful; after the loop I waited about three weeks before getting a final update. In the end, I received a “No Offer”

decision due to team fit/headcount constraints according to the recruiter. The recruiter said my DSA performance was solid but I needed more depth in system design and more concrete examples tied to certain Leadership Principles like Customer Obsession. Despite the rejection, the experience matched what I saw on Glassdoor: a structured but demanding process, heavy emphasis on coding, online assessments, and the Leadership Principles. It gave me a clear sense of Amazon's expectations and helped me identify gaps in my system design and behavioral storytelling. experience

3. (interview on may 21.

Round 1: 2 simple coding question with a couple behavioral. The coding was super easy, I was quite nervous but ended up getting the optimal solution for the first one and got a solution on the second just didn't have time to write it out. (Both questions were essentially two pointer)

Round 2: Bar Raiser - Senior SWE and a Manager sitting in and it was just all behavioral stayed energetic and positive throughout. Rambled on some questions but was able to clarify and get to the point this round went well for me.

Round 3: One coding question and a behavioral, Honestly, I didn't understand the question at all when it was given to me and I implemented a solution that was correct, but didn't account for a specific edge case. The interviewer was super quiet. I walked him through my solution and ran some dry cases talking throughout but he wasn't saying a word just gave me an edge case and asked how I would move forward with that. Honestly, not sure how I feel about that round but the questioning afterwards was quite positive and felt super conversational. Interviewer was super nice they all were his calm demeanor was semi intimidating but whatever LOL (DP Problem, its straight from leetcode and is genuinely quite difficult I wont even lie to you guys). Unfortunately I will not be saying what question it is incase that somehow impacts my chances at an offer but think money.

Round 4: LLD round + Behavioral. LLD was super simple to start and as the follow ups kept getting added on it got more tricky. But at the end of the day I was explaining everything and my thought process, the interviewer was able to follow along with what I was thinking and while the syntax and everything was good to a point he mentioned that he got my mental model down and that's all that he was really looking for. We had a conversation afterwards, felt good and he said qualities he looks for he saw in me so that's fire. When it comes to questions like this, just think of a real world example, and work backwards.

)

4

 Virtual On-site Interviews

After successfully passing the OA, I received an email from the Amazon University Talent Acquisition(AUTA) team asking to schedule a time to conduct the virtual on-site interviews. The next step

[was a virtual onsite, consisting of three 1-hour rounds.](#)

[On the day of the interview, I got an email mentioning that: due to a last-minute conflict, your 1st interview for today will be rescheduled to another date. Please proceed with the other 2 rounds as scheduled, and please share your availability for the next 2 weeks so that we can reschedule your missed interview.](#)

[Press enter or click to view image in full size](#)

[Later, I got my first coding round conducted 3 days after receiving the above email. Here's the detailed breakdown of the interview rounds:](#)

[■ Round 1: Coding + Object-Oriented Design](#)

[Format:](#)

[Duration: 1 hour long](#)

[Environment: Amazon Chime + Amazon LiveCode](#)

[Structure: 1 OOD design question + a short discussion toward the end](#)

[This round focused on designing a system using object-oriented design\(OOD\)/Low-Level Design\(LLD\) principles. I had to identify the right classes, define appropriate attributes and methods, and ensure that all constraints were enforced \(like limits on resource usage, access control, and tracking\).](#)

[What this round tested:](#)

[Object-oriented design\(Low Level design\) thinking](#)

[Modelling real-world constraints into clean class structures](#)

[Effective use of inheritance, encapsulation, and data validation](#)

[What Helped Me Succeed:](#)

[Practicing system design questions involving entities, rules, and relationships](#)

[Keeping my code readable and modular](#)

[Thinking out loud and confirming my design assumptions with the interviewer](#)

[■ Round 2: Data Structures & Logic Implementation](#)

[Format:](#)

[Duration: 1 hour long](#)

[Environment: Amazon Chime + Amazon LiveCode](#)

[Structure: 2 coding questions + a short discussion toward the end](#)

[This round had two back-to-back questions:](#)

[Question 1:](#)

[A real-time simulation of data ingestion and querying was involved. I was expected to build the system logic to efficiently store\(selecting an efficient data structure, e.g.. I used OrderedDict in Python in this round\) and query incoming stream updates. There were additional requirements like aggregation, top-K selection, and maintaining historical snapshots.](#)

[Get Tharun Kumar Reddy Polu's stories in your inbox](#)

[Join Medium for free to get updates from this writer.](#)

[Enter your email](#)

[Subscribe](#)

[Remember me for faster sign in](#)

[What it tested:](#)

[Use of hash maps and lists for structured time-series data](#)

[Writing efficient queries with filtering and sorting](#)

[Edge case handling for missing or invalid input](#)

[Question 2:](#)

[This question focused on efficient item packing, where items with certain weights needed to be grouped optimally without exceeding capacity. The problem demanded both greedy and analytical thinking.](#)

[What Helped Me:](#)

[Explaining my thought process before coding](#)

[Walking through custom test cases to validate edge behaviour](#)

[Clearly stating time and space complexity at the end](#)

[👤👤 Round 3: Managerial / Bar Raiser Interview](#)

Format:

Duration: 1 hour long

Environment: Amazon Chime

Structure: Amazon Leadership Principles, Behavioral questions, values alignment

This was a behavioral deep dive where I was expected to discuss my past work, decision-making process, and how I handle complexity. The interviewer followed up on each answer with layered questions to gauge depth and self-awareness.

Areas accessed during this round:

How I take ownership in ambiguous situations

How I respond to failure or constructive feedback

How do I make trade-offs between fast execution and long-term quality

What drives me to build scalable and meaningful systems

What Made It Challenging:

Midway through the interview, I found myself mentally saturated. The expectations for depth and reflection were high, and I had already spoken about my most impactful projects. It taught me how important mental energy management is during interviews.

What Helped Me:

Structuring every response using the STAR method

Being honest and not trying to “make up” the answers

Sharing both wins and learnings from failures I faced

Tying experiences back to impact, ownership, and long-term growth

Talking about the feedback I received from mentors and how I improved

💡 Lessons learned & Tips for You

What Worked:

Practicing real interview-style problems under timed conditions

Able to come up with a good object-oriented design under pressure

LeetCode (Amazon Tagged + Blind 75 + Grokking Coding Patterns)

Clear explanation of Time and space complexity tradeoffs, not just brute force solutions

Key Takeaways from my viewpoint:

Object-oriented design interviews are more about thinking clearly than writing perfect code

In coding rounds, testing your code thoroughly(without being prompted by the interviewer) can leave a strong impression

Managerial interviews demand deep introspection — prepare 3–4 strong stories and know them well

Even if you perform well, some rounds may weigh more than others. Don't take rejection personally

Most importantly, never stop reflecting and refining your approach

Works cited

1. Step-by-Step Guide for FAANG Interviews in 2026 (Part 5 of 10): The Master Roadmap, accessed on May 21, 2026,
<https://medium.com/@emilyhustlenyc/how-to-pass-the-amazon-software-engineer-interview-in-2026-leadership-principles-the-bar-raiser-c03901244baa>
2. Amazon Software Development Engineer (SDE) New Grad Interview Review | LockedIn AI, accessed on May 21, 2026,
<https://www.lockedinai.com/company-details/AMZN/9f458ad0-e6e0-4c6b-bc21-4bcde70ce1c3>
3. Interview Loop - Amazon Careers, accessed on May 21, 2026,
<https://amazon.jobs/content/en/how-we-hire/interview-loop>
4. Amazon SDE 1 Interview Experience (2026) : r/amazonsdeprep, accessed on May 21, 2026,
https://www.reddit.com/r/amazonsdeprep/comments/1s8iqxq/amazon_sde_1_interview_experience_2026/
5. Amazon Interview Experience & Timeline (Almost a 1-Year Process) | 2025 Grad - Reddit, accessed on May 21, 2026,
https://www.reddit.com/r/amazonemployees/comments/1ruve39/amazon_interview_experience_timeline_almost_a/
6. 7 months with Amazon | New Grad/SDE I | Seattle | Reject | August 2020 - March 2021 - Discuss - LeetCode, accessed on May 21, 2026,
<https://leetcode.com/discuss/interview-experience/1134984/7-months-with-amazon-new-gradsde-i-seattle-reject-august-2020-march-2021/>
7. Amazon SDE 1 (USA) 2026 Hiring – Complete Timeline & Updates Thread : r/leetcode - Reddit, accessed on May 21, 2026,
https://www.reddit.com/r/leetcode/comments/1r9ieoz/amazon_sde_1_usa_2026_hiring_complete_timeline/
8. Amazon Onsite Interview 2026 SDE1 : r/amazonemployees - Reddit, accessed on May 21, 2026,
https://www.reddit.com/r/amazonemployees/comments/1qh4a43/amazon_onsite_interview_2026_sde1/
9. My Amazon Software Development Engineer New Grad Interview Experience - Stackademic, accessed on May 21, 2026,
<https://blog.stackademic.com/my-amazon-software-development-engineer-new-grad-interview-experience-f16d4ccba896>

10. Amazon SDE-1 New Grad - US : r/leetcode - Reddit, accessed on May 21, 2026, https://www.reddit.com/r/leetcode/comments/1lrs5nk/amazon_sde1_new_grad_us/
11. Amazon behavioral interview questions | Free cheat sheet - Hustle Badger, accessed on May 21, 2026, <https://www.hustlebadger.com/what-do-product-teams-do/amazon-behavioral-interview-questions/>
12. A Complete Guide to the Amazon Bar Raiser Interview Process, accessed on May 21, 2026, <https://www.scarletink.com/a-complete-guide-amazon-bar-raiser/>
13. How to Get Job in Amazon: Step-by-Step Guide 2026 - upGrad, accessed on May 21, 2026, <https://www.upgrad.com/blog/amazon-jobs-how-to-get-a-job-at-amazon/>
14. Performance Review - Amazon - Taro, accessed on May 21, 2026, <https://www.jointaro.com/topic/amazon/performance-review/>
15. 90% of you won't get through the application process and it's not because of your technical skills : r/degreeapprenticeships - Reddit, accessed on May 21, 2026, https://www.reddit.com/r/degreeapprenticeships/comments/1rs5rwd/90_of_you_wont_get_through_the_aapplication/
16. Amazon Behavioral Interview Questions (+ answers, method ..., accessed on May 21, 2026, <https://igotanooffer.com/blogs/tech/amazon-behavioral-interview>
17. Amazon Interview Coaching, accessed on May 21, 2026, <https://amazonbound.today/amazon-interview-coach>
18. Amazon AI Engineer Interview Guide 2026 - Dataford, accessed on May 21, 2026, <https://dataford.io/interview-guides/amazon/ai-engineer>
19. Amazon Bound | Nick Dimitrov, accessed on May 21, 2026, <https://amazonbound.podbean.com/>
20. How to Master Amazon Interview Follow-Up Questions | Day One Careers, accessed on May 21, 2026, <https://dayone.careers/how-to-master-interview-follow-up-questions>
21. 30-Day Job Search Success Guide | PDF | Résumé | Leadership - Scribd, accessed on May 21, 2026, <https://www.scribd.com/document/973231159/Abhijay-Job-Hunting-Master-Guide-1>
22. The Amazon Behavioral Interview - GitHub, accessed on May 21, 2026, <https://github.com/tombetthausen/amazon-behavioral-interview>
23. Amazon STAR Interview Questions for SDEs: 15 Answers to Prepare ..., accessed on May 21, 2026, <https://www.vervecopilot.com/interview-questions/top-30-most-common-amazon-star-interview-questions-you-should-prepare-for>
24. Amazon Behavioral Interview Questions and Answers (2026 Guide) - Exponent, accessed on May 21, 2026, <https://www.tryexponent.com/blog/how-to-nail-amazons-behavioral-interview-questions>
25. Top 30 Most Common Amazon Behavioral Interview Questions You Should Prepare For - Verve AI, accessed on May 21, 2026, <https://www.vervecopilot.com/interview-questions/top-30-most-common-amazon-behavioral-interview-questions-you-should-prepare-for>
26. Amazon Bar Raiser Interview (questions, prep tips) - IGotAnOffer, accessed on May 21, 2026, <https://igotanooffer.com/blogs/tech/amazon-bar-raiser-interview>
27. Amazon Bias for Action Interview Questions (+ How to Answer Them ..., accessed on May 21, 2026, <https://igotanooffer.com/en/advice/amazon-bias-for-action-interview-questions>
28. Amazon Behavioral Interview Questions 2025 - Design Gurus, accessed on May 21, 2026, <https://www.designgurus.io/blog/amazon-behavioral-interview-questions>
29. Amazon Behavioral Interview Questions Guide - Careerflow.ai, accessed on May 21, 2026, <https://www.careerflow.ai/blog/amazon-behavioral-interview-questions>

30. My Amazon SDE-1 Interview Experience | by Sarvesh Mankar - Medium, accessed on May 21, 2026, <https://medium.com/@SarveshMankar/my-amazon-sde-1-interview-experience-ed728372c5d0>
31. About Us - Amazon's Leadership Principles, accessed on May 21, 2026, <https://www.aboutamazon.com/about-us/leadership-principles>
32. Help with Amazon Behavioral Interview Stories. : r/leetcode - Reddit, accessed on May 21, 2026, https://www.reddit.com/r/leetcode/comments/1jc9qom/help_with_amazon_behavioral_interview_stories/
33. AWS Front End Interview Series: From Application to Phone Screen — Part 1 - Medium, accessed on May 21, 2026, <https://medium.com/@ritikajoshi141/aws-front-end-interview-series-from-application-to-phone-screen-part-1-of-2-8bd24350fc41>
34. yashchoubey/interview - GitHub, accessed on May 21, 2026, <https://github.com/yashchoubey/interview>
35. Amazon Behavioral Interview: Overview, Examples & Prep Tips - JobTestPrep, accessed on May 21, 2026, <https://www.jobtestprep.com/amazon-behavioral-interview>
36. Top 10 Amazon Behavioral Interview Questions (With Winning Answers That Actually Work + Insider Tips), accessed on May 21, 2026, <https://blog.theinterviewguys.com/amazon-behavioral-interview-questions/>
37. The definitive guide to cracking an Amazon business interview | by Asik Das | Medium, accessed on May 21, 2026, <https://medium.com/@asikdas/the-definitive-guide-to-cracking-an-amazon-business-interview-71cafa71c1e8>
38. Preparing for Behavior Based Interview Questions - Fortify Experts, accessed on May 21, 2026, <https://fortifyexperts.com/preping-for-behavior-based-interview-questions/>
39. Amazon Behavioral Interview Questions and Answers | How to Pass the Interview - YouTube, accessed on May 21, 2026, <https://www.youtube.com/watch?v=gjfldnuQrcg>
40. Unable to hire people on our team due to endless worship of LPs during interviews. - Reddit, accessed on May 21, 2026, https://www.reddit.com/r/amazonemployees/comments/1mndpjr/unable_to_hire_people_on_our_team_due_to_endless/
41. Master The Amazon Interview Loop: Proven Strategies from a Former Bar Raiser - YouTube, accessed on May 21, 2026, <https://www.youtube.com/watch?v=RnZULkgBpfU>
42. How Amazon's debrief process ensures hiring the best - Working ..., accessed on May 21, 2026, <https://workingbackwards.com/blog/how-amazons-debrief-process-sures-hiring-the-best/>
43. Three Tales from an Amazon Bar Raiser - by Dave Anderson, accessed on May 21, 2026, <https://www.scarletink.com/three-theses-from-an-amazon-bar-raiser/>
44. How do I impress the F out of the Bar Raiser? : r/amazonemployees - Reddit, accessed on May 21, 2026, https://www.reddit.com/r/amazonemployees/comments/1lfhche/how_do_i_impress_the_f_out_of_the_bar_raiser/